

CRYPTOGRAPHY BASICS

[Kaynak](#)

İÇERİK

1. Kriptografinin Önemi ve Kullanım Alanları
2. Plaintext'ten Ciphertext'e: Şifreleme Sürecinin Anatomisi
3. Tarihsel Şifreleme Yöntemleri: Caesar Cipher ve Ötesi
4. Şifreleme Türleri: Simetrik ve Asimetrik Kriptografi
5. Kriptografinin Temel Matematiği

Kriptografinin Önemi ve Kullanım Alanları

Kriptografi (şifreleme bilimi), en temel haliyle **adversaries** (hasımların/saldırganların) bulunduğu bir ortamda güvenli iletişimi sağlama sanatıdır. Buradaki "güvenlik" kavramı sadece verinin okunmasını engellemek değil, aynı zamanda verinin değiştirilmediğinden emin olmayı da kapsar.

Kriptografinin Temel Amaçları

Bir siber güvenlik uzmanı olarak bilmen gereken en kritik nokta şudur: Kriptografi sadece "şifreleme" demek değildir; üç ana sütun üzerine inşa edilir:

1. **Confidentiality (Gizlilik)**: Verinin yetkisiz kişiler tarafından okunmasını engellemek.
2. **Integrity (Bütünlük)**: Verinin iletim sırasında veya depolanırken değiştirilmediğinden, bozulmadığından emin olmak.
3. **Authenticity (Kimlik Doğrulama/Özgünlük)**: İletişim kurduğumuz kişinin gerçekten iddia ettiği kişi olduğunu doğrulamak.

Günlük Hayattan Teknik Senaryolar

Kriptografiyi her gün farkında olmadan kullanıyoruz. Bu süreçlerin arka planında dönen mantığı anlamak, saldırı vektörlerini analiz ederken bize yardımcı olur:

1. Kimlik Bilgilerinin Korunması (Login Süreçleri)

TryHackMe veya başka bir platforma giriş yaparken kullanıcı adı ve parolanız ağ üzerinden **cleartext** (açık metin) olarak gitmez. Eğer gitseydi, ağdaki bir saldırgan **Packet Sniffing** (paket koklama) yöntemiyle bu bilgileri kolayca ele geçirebilirdi. Kriptografi sayesinde bu veriler şifrelenerek sunucuya iletilir.

2. Güvenli Uzak Bağlantı (SSH)

SSH (Secure Shell) üzerinden bir sunucuya bağlandığınızda, istemciniz ve sunucu arasında **Encrypted Tunnel** (şifreli tünel) kurulur. Bu tünel sayesinde, oturumunuz boyunca gönderdiğiniz tüm komutlar ve aldığınız yanıtlar üçüncü taraflarca dinlenemez hale gelir (**Eavesdropping** koruması).

3. Sertifika ve Güven İlişkisi (Online Bankacılık)

Banka sitesine girdiğinizde tarayıcınız sunucunun sertifikasını kontrol eder. Bu işlem, sadece veriyi şifrelemekle kalmaz; aynı zamanda "Ben gerçekten bankanın sunucusuyla mı konuşuyorum, yoksa araya girmiş bir saldırganla mı?" sorusuna yanıt verir.

4. Veri Doğrulama (Hash Fonksiyonları)

Büyük bir dosya indirdiğinizde, dosyanın eksik veya hatalı inip inmediğini anlamak için **Hash** fonksiyonları kullanılır. Kriptografik bir hash, verinin parmak izidir. Eğer orijinal dosyanın hash değeri ile sizin indirdiğiniz dosyanın hash değeri birbirine eşitse, veri bütünlüğü (**Integrity**) korunmuş demektir.

Yasal Uyumluluk ve Standartlar (Compliance)

Kriptografi sadece teknik bir tercih değil, çoğu zaman yasal bir zorunluluktur. Şirketlerin veriyi hem **At Rest** (depolanırken/disk üzerinde) hem de **In Motion** (iletim halindeyken/ağ üzerinde) şifrelemesi gerekir.

PCI DSS (Payment Card Industry Data Security Standard)

Kredi kartı bilgilerini işleyen, saklayan veya ileten her kuruluş bu standarta uymak zorundadır. **PCI DSS**, kart verilerinin hem veritabanında saklanırken hem de sunucular arası transfer edilirken şifrelenmiş olmasını şart koşar.

Sağlık Verileri ve Bölgesel Yasalar

Tıbbi kayıtların korunması, kredi kartlarına göre daha karmaşık ve ülkeye göre değişkenlik gösteren yasalara tabidir:

- **ABD:** HIPAA (Health Insurance Portability and Accountability Act) ve HITECH.
- **Avrupa Birliği:** GDPR (General Data Protection Regulation).
- **Birleşik Krallık:** DPA (Data Protection Act).

Özetle: Bir sistem tasarlarken veya bir sızma testi yaparken, verinin sadece nasıl şifrelendiğine değil, hangi yasal regülasyonlara (Compliance) tabi olduğuna da bakmalısın. Kriptografi, kullanıcıdan gizlenen ama sistemin her katmanında bulunması gereken bir zorunluluktur.

Plaintext'ten Ciphertext'e: Şifreleme Sürecinin Anatomisi

Kriptografinin temel işleyişini anlamak için verinin bir formdan diğerine nasıl dönüştüğünü ve bu süreçte kullanılan temel yapı taşlarını bilmek gerekir. Bu süreç, sadece metinleri değil, dijital dünyadaki her türlü veriyi (görsel, ses, binary veri vb.) kapsar.

Temel Kavramlar Sözlüğü

Bir siber güvenlik uzmanı olarak bu terimleri sadece bilmek yetmez, aralarındaki ilişkiyi mantıksal bir düzleme oturtman gerekir:

- **Plaintext (Açık Metin):** Şifrelenmemiş, doğrudan okunabilir ve anlamlı veridir. Bu bir "merhaba" mesajı, bir kedi fotoğrafı, kredi kartı numarası veya bir veritabanı dosyası olabilir. Kriptografi perspektifinden bakıldığında, işlenmemiş her veri "plaintext"tir.
- **Ciphertext (Şifreli Metin):** Şifreleme işlemi sonucunda ortaya çıkan, karmaşık ve okunamaz haldeki veridir. İdeal bir ciphertext, orijinal mesaj hakkında (yaklaşık boyutu dışında) hiçbir ipucu vermemelidir.
- **Cipher (Şifreleme Algoritması):** Veriyi açık metinden şifreli metine (ve tam tersine) dönüştürmek için kullanılan matematiksel yöntem veya algoritmadır. Genellikle matematikçiler tarafından geliştirilir ve güvenilirliği kanıtlanmış olanlar kamuya açıktır.
- **Key (Anahtar):** Algoritmanın (cipher) veriyi işlerken kullandığı bit dizisidir (0 ve 1'lerden oluşan dizi). Algoritmanın kendisi herkes tarafından bilinse bile, anahtar gizli tutulduğu sürece veri güvendedir.

Not: Asimetrik şifrelemede durum biraz farklıdır (Public Key kavramı), ancak genel kural anahtarın gizliliğidir.

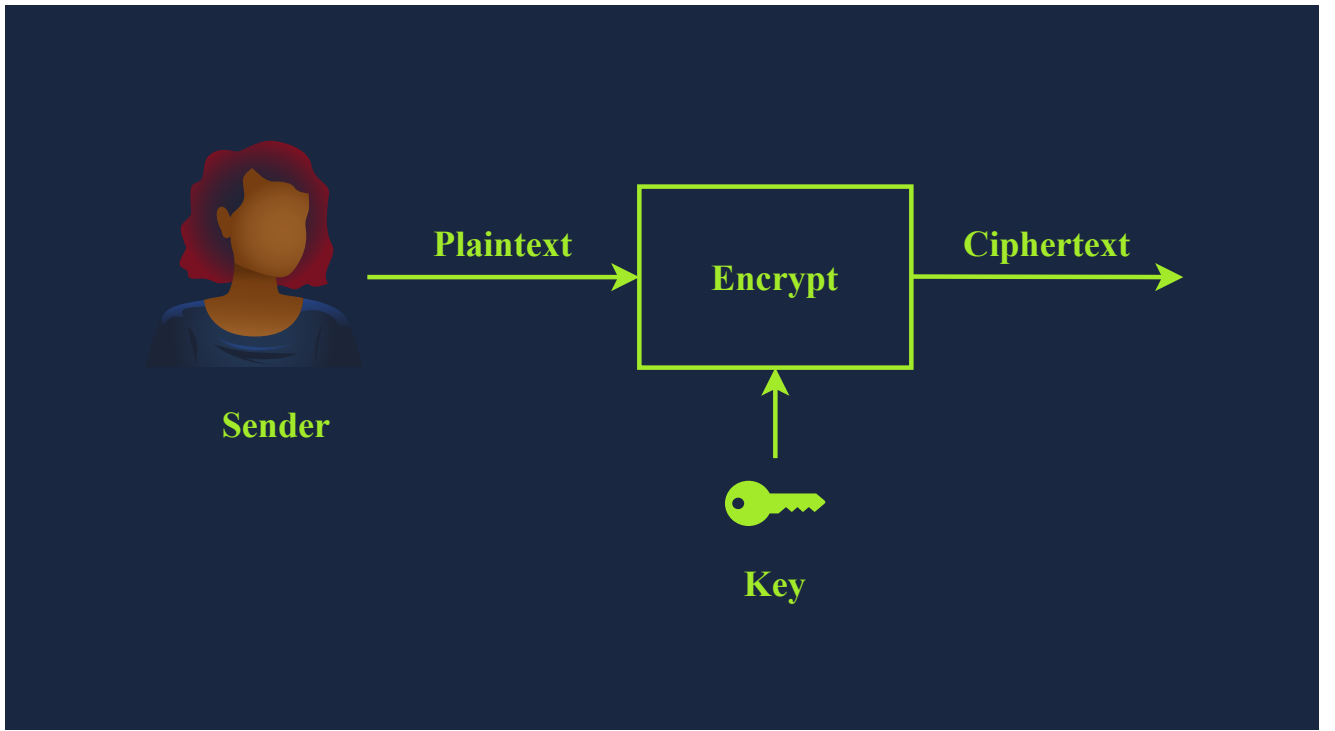
- **Encryption (Şifreleme):** Plaintext'in bir cipher ve bir key kullanılarak ciphertext'e dönüştürülmesi sürecidir.
- **Decryption (Şifre Çözme):** Ciphertext'in uygun anahtar ve algoritma kullanılarak yeniden orijinal plaintext haline getirilmesi sürecidir.

Şifreleme Mantığı ve İşleyiş Süreci

Süreci bir fonksiyon olarak düşünürsek, mantık zinciri şu şekilde ilerler:

1. Şifreleme Aşaması

Elinizde bir veri (**Plaintext**) ve bu veriyi kilitleyecek bir mekanizma (**Cipher**) vardır. Ancak bu mekanizmanın çalışması için size özel bir değişkene (**Key**) ihtiyaç duyar.

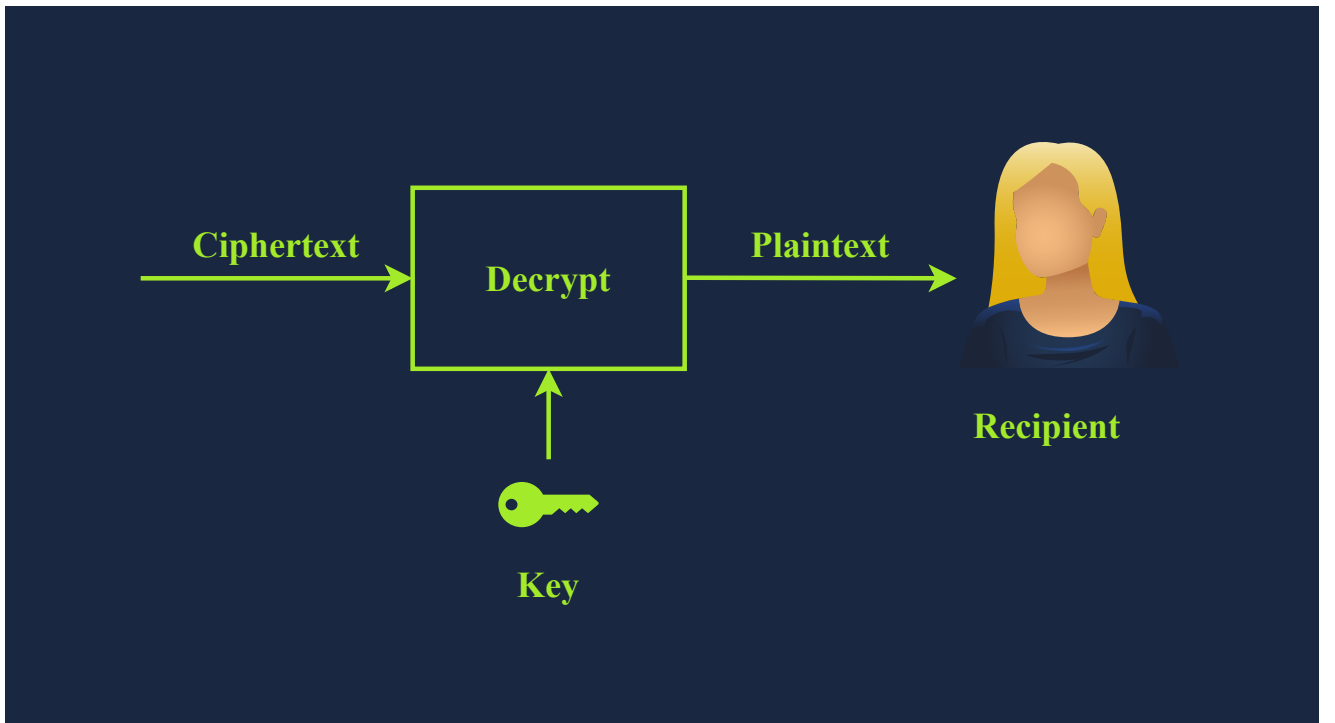


Neden bir anahtara ihtiyaç duyarız?

Eğer sadece algoritma (cipher) kullansaydık, algoritmayı bilen herkes şifreyi çözebilirdi. Anahtar sayesinde, aynı algoritmayı kullanan milyonlarca insan olsa bile, her birinin şifreli metni birbirinden farklı ve sadece kendi anahtarlarıyla açılabilir olur.

2. Şifre Çözme Aşaması

Şifreli metni orijinal haline döndürmek için sürecin tam tersi işletilir. Burada kritik nokta, doğru anahtarın kullanılmasıdır.



Kritik Güvenlik Prensipleri

1. **Algoritma vs. Anahtar Gizliliği:** Modern kriptografide "algoritmayı gizli tutarak güvenlik sağlama" (security by obscurity) yöntemi zayıf kabul edilir. Güvenilir bir sistemde algoritma herkes tarafından bilinebilir olmalı, ancak anahtara sahip olmayan biri için şifreyi çözmek **matematiksel olarak imkansız veya pratik olarak imkansız** (computationally infeasible) olmalıdır.
2. **Veri Türü Bağımsızlığı:** Kriptografi için verinin ne olduğu önemli değildir. Bir .jpg dosyası ile bir .txt dosyası şifreleme fonksiyonuna girdiğinde, fonksiyon bunları sadece **binary** (ikili) veri olarak görür ve işler.

Tarihsel Şifreleme Yöntemleri: Caesar Cipher ve Ötesi

Kriptografinin kökenleri M.Ö. 1900'lü yıllara, Eski Mısır'a kadar uzanır. Ancak modern kriptografi mantığını anlamak için en temel ve klasik örnek, M.Ö. 1. yüzyılda kullanılan **Caesar Cipher** (Sezar Şifrelemesi) yöntemidir.

Caesar Cipher (Sezar Şifrelemesi)

Bu yöntem, her harfin alfabede belirli bir sayı kadar kaydırılması esasına dayanır. Bu kaydırma miktarı bizim **Key** (anahtar) değerimizdir.

1. Şifreleme Süreci (Encryption)

Bir mesajı şifrelemek için her harfi alfabede sağa doğru belirtilen anahtar kadar ilerletiriz.

- **Plaintext:** TRYHACKME
- **Key (Shift):** 3 (Sağa doğru 3 kaydırma)
- **İşlem:** $T + 3 \rightarrow W$
 - $R + 3 \rightarrow U$
 - $Y + 3 \rightarrow B$ (Z'den sonra alfabenin başına dönülür - *Modular Arithmetic*)
 - ...bu şekilde devam eder.
- **Ciphertext:** WUBKDFNPH

2. Şifre Çözme Süreci (Decryption)

Şifreyi çözmek için yapılan işlemin tam tersini, yani anahtar kadar sola kaydırma işlemini uyguluyoruz.

- **Ciphertext:** WUBKDFNPH
- **Key:** 3 (Sola doğru 3 kaydırma)
- **Sonuç:** TRYHACKME

Güvenlik Analizi: Neden Güvensiz?

Bir siber güvenlik uzmanı gözüyle baktığımızda, Caesar Cipher günümüz standartlarında tamamen **Insecure** (güvensiz) kabul edilir. Bunun temel sebebi **Key Space** (anahtar uzayı) dediğimiz olasılık kümesinin çok küçük olmasıdır.

- **Zayıf Nokta:** İngiliz alfabesi 26 harften oluşur. Bir harfi 26 kez kaydırırsanız harf kendisine döner. Bu da toplamda sadece **25 olası anahtar** olduğu anlamına gelir.
- **Brute Force Saldırısı:** Bir saldırgan elindeki şifreli metni (ciphertext) alıp 25 farklı anahtarı da denerse, anlamlı metni bulması sadece saniyeler (hatta manuel olarak dakikalar) sürer. Eğer bir metin Sezar ile şifrelendiyse, anahtarı bilmeseniz bile tüm ihtimalleri denemek (exhaustive search) kesin sonuç verir.

Diğer Önemli Tarihsel Şifreler

Kriptografi tarihi boyunca Sezar'dan daha karmaşık yöntemler de geliştirilmiştir. Bunlar sızma testleri veya CTF (Capture The Flag) yarışmalarında sıkça karşımıza çıkar:

1. **Vigenère Cipher (16. Yüzyıl):** Sezar'ın aksine, her harf için farklı bir kaydırma değeri kullanır. Bu değere bir anahtar kelime (keyword) aracılığıyla karar verilir. (Polialfabetik şifreleme).
2. **Enigma Machine (2. Dünya Savaşı):** Nazi Almanyası tarafından kullanılan, dönen diskler (rotorlar) yardımıyla her harfi karmaşık bir şekilde şifreleyen elektromekanik bir cihazdır. Alan Turing ve ekibinin bu şifreyi kırması modern bilgisayar biliminin temellerini atmıştır.
3. **One-Time Pad (Soğuk Savaş):** Eğer anahtar tamamen rastgele ise, mesajla aynı boyuttaysa ve **asla** ikinci kez kullanılmıyorsa, bu yöntem matematiksel olarak kırılmaz (Unbreakable) kabul edilir.

Pratik Araçlar

Bu tür klasik şifreleme yöntemlerini hızlıca test etmek veya CTF sorularında çözmek için [Cryptii](#) gibi web tabanlı araçlar kullanılır. Bu araçlarda rotasyon (ROT) miktarını anlık olarak değiştirerek metnin nasıl dönüştüğünü görebilirsin.

Kritik Not: Modern sistemlerde (HTTPS, SSH vb.) artık bu tarihsel yöntemler kullanılmaz; bunların yerini çok daha karmaşık matematiksel temellere dayanan AES gibi algoritmalar almıştır.

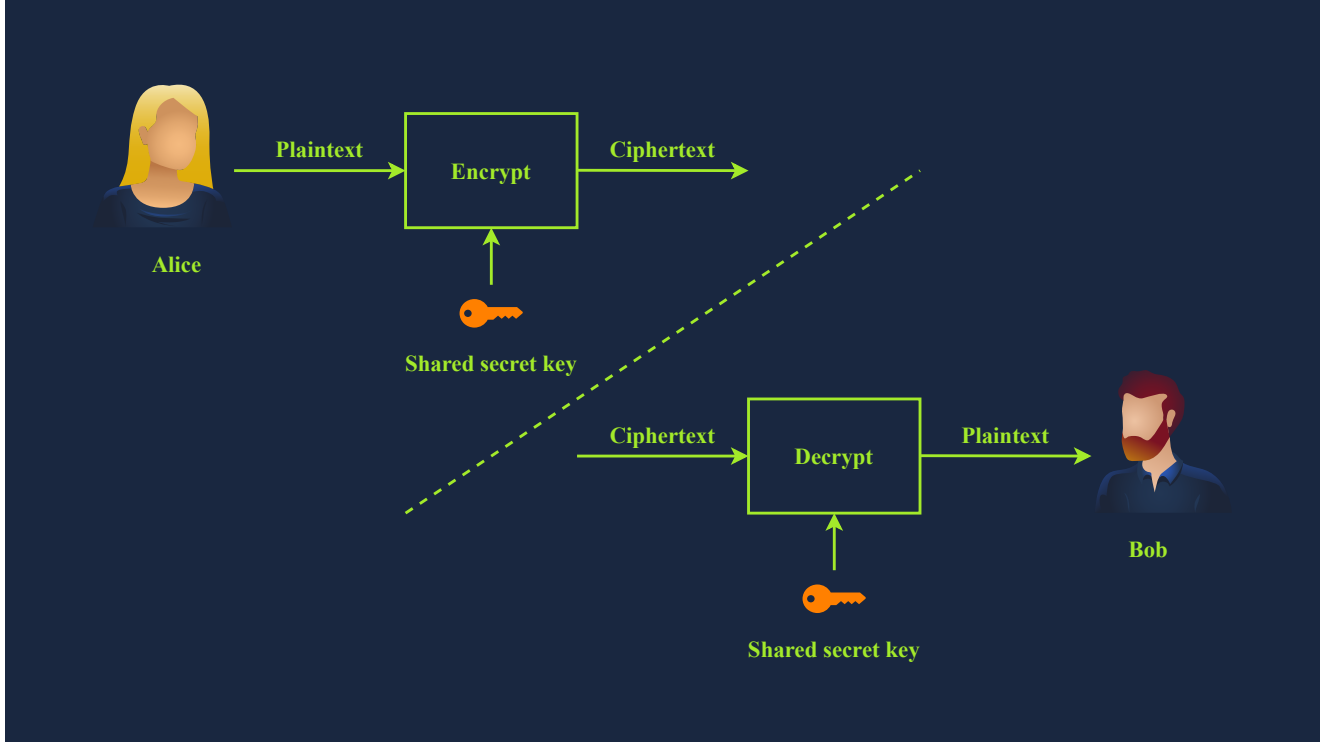
Şifreleme Türleri: Simetrik ve Asimetrik Kriptografi

Modern bilişim dünyasında veriyi korumak için kullandığımız yöntemler iki ana kategoriye ayrılır: **Symmetric** (Simetrik) ve **Asymmetric** (Asimetrik) şifreleme. Bu iki yöntem arasındaki farkı anlamak, ağ güvenliği ve sistem mimarisini kavramak için hayati önem taşır.

1. Simetrik Şifreleme (Symmetric Encryption)

Simetrik şifrelemede veriyi şifrelemek (encrypt) ve şifresini çözmek (decrypt) için **aynı anahtar** kullanılır. Bu nedenle bu yönetime **Private Key Cryptography** (Özel Anahtar

Kriptografisi) de denir.



Temel Özellikler ve Zorluklar

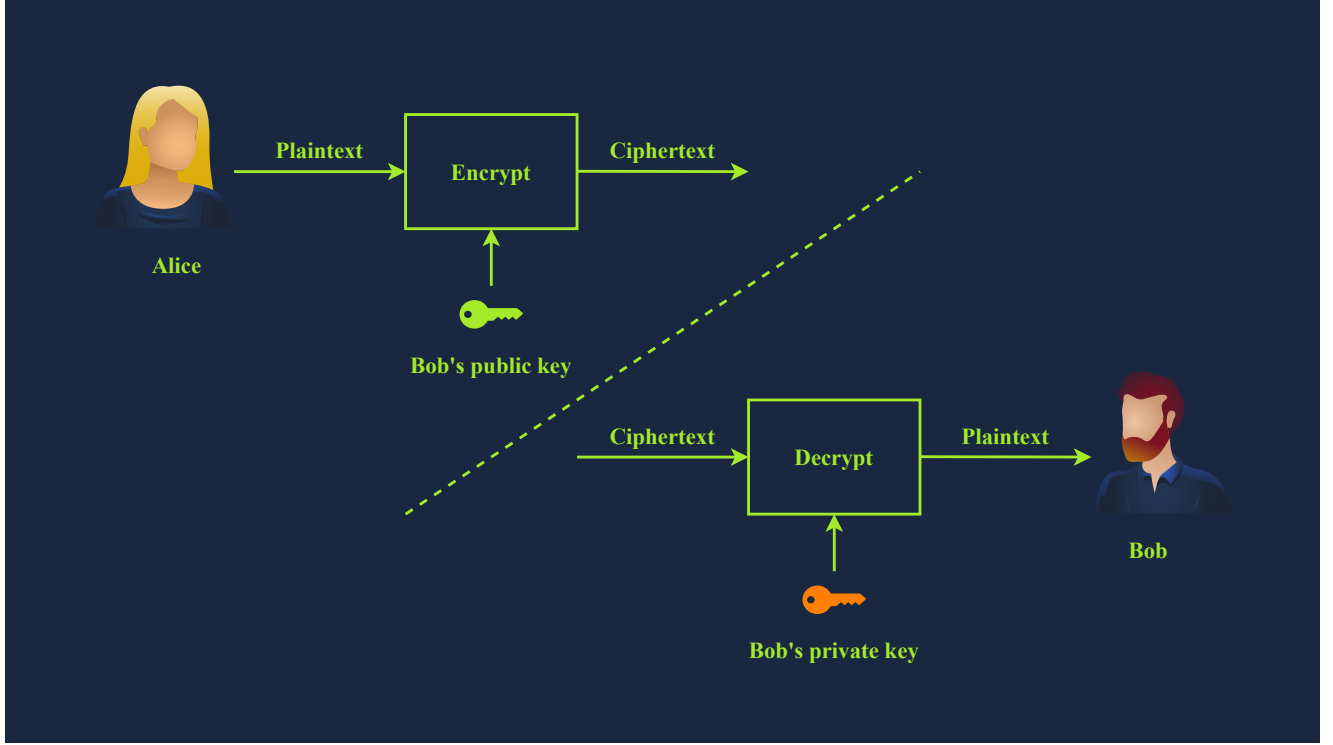
- **Hız:** Matematiksel olarak daha az karmaşık olduğu için asimetrik şifrelemeye göre çok daha hızlıdır. Büyük veri yığınlarını şifrelemek için idealdir.
- **Anahtar Dağıtım Problemi (Key Exchange):** En büyük zayıflığıdır. Eğer iş arkadaşına şifreli bir belge gönderiyorsan, şifreyi (anahtarı) de ona ulaştırman gerekir. Şifreyi e-posta ile gönderirsen, e-postayı ele geçiren biri belgeyi de açabilir. Bu yüzden anahtarı paylaşmak için **güvenli bir kanal** (yüz yüze görüşme veya farklı bir güvenli iletişim yolu) gerekir.
- **Ölçeklenebilirlik:** Alıcı sayısı arttıkça anahtarların gizliliğini korumak zorlaşır.

Yaygın Simetrik Algoritmalar

Algoritma	Durumu	Açıklama
DES	Güvensiz	1977 standardı. 56-bit anahtar kullanır. 1999'da 24 saatten kısa sürede kırıldığı için artık kullanılmaz.
3DES	Legacy (Eski)	DES'in üç kez uygulanmış halidir. 112-bit efektif güvenlik sağlar. 2019'da kullanımdan kaldırılmıştır (deprecated), sadece eski sistemlerde rastlanır.
AES	Standart	2001'den beri dünya standardıdır. 128, 192 ve 256-bit anahtar boyutlarını destekler. Günümüzde en güvenli ve yaygın olanıdır.

2. Asimetrik Şifreleme (Asymmetric Encryption)

Simetrik yöntemin aksine, burada birbirine matematiksel olarak bağlı **bir anahtar çifti** kullanılır. Bu yöneme **Public Key Cryptography** (Açık Anahtar Kriptografisi) denir.



- **Public Key (Açık Anahtar):** Herkese açık olabilir, herkesle paylaşılabilir. Veriyi **şifrelemek** için kullanılır.
- **Private Key (Özel Anahtar):** Sadece sahibinde kalmalıdır, asla paylaşılmaz. Şifrelenmiş veriyi **çözmek** için kullanılır.

Mantık Zinciri: Alice, Bob'a gizli bir mesaj göndermek isterse; Bob'un **Public Key**'ini alır ve mesajı bununla şifreler. Bu mesajı artık sadece Bob kendi **Private Key**'i ile açabilir. Yolda mesaj çalınrsa bile, saldırganda Bob'un özel anahtarı olmadığı için mesajı okuyamaz.

Algoritmalar ve Anahtar Boyutları

Asimetrik şifreleme, tek yönde hesaplaması kolay ama tersine çevrilmesi (kırılması) "matematiksel olarak imkansız" (milyonlarca yıl sürecek) problemlere dayanır.

- **RSA:** En yaygın asimetrik algoritmadır. Güvenli kabul edilmesi için en az **2048-bit** anahtar kullanmalıdır (3072 ve 4096-bit daha güvenlidir).
- **Diffie-Hellman:** Genellikle güvenli anahtar değişimi için kullanılır. Önerilen minimum boyut **2048-bit**'tir.
- **ECC (Elliptic Curve Cryptography):** Eliptik eğri matematiğini kullanır. En büyük avantajı, çok daha küçük anahtar boyutlarıyla yüksek güvenlik sunmasıdır. Örneğin; **256-bit ECC** anahtarı, **3072-bit RSA** anahtarına eşdeğer güvenlik sağlar. Bu da hız ve performans avantajı demektir.

Özet Karşılaştırma Tablosu

Özellik	Simetrik Şifreleme	Asimetrik Şifreleme
Anahtar Sayısı	Tek anahtar (Aynı anahtar her iki işi de yapar).	İki anahtar (Public ve Private).
Hız	Çok hızlı.	Daha yavaş.
Kullanım Amacı	Büyük verilerin şifrenmesi.	Anahtar değişimi, dijital imzalar.
Anahtar Paylaşımı	Zor (Gizli kanal gerekir).	Kolay (Public key herkese verilebilir).

Kriptografinin Temel Matematiği

Modern kriptografinin kalbi matematiktir. Karmaşık algoritmaların (AES, RSA vb.) derinliklerine indiğimizde karşımıza çıkan en temel iki yapı taşı **XOR** ve **Modulo** işlemleridir. Bunları anlamadan kriptografik bir zafiyeti analiz etmek imkansızdır.

1. XOR İşlemi (Exclusive OR - Özel VEYA)

XOR, ikili (binary) aritmetikte kullanılan mantıksal bir işlemdir. Kriptografide bu kadar popüler olmasının sebebi, **tersine çevrilebilir** ve **hızlı** olmasıdır. Genellikle veya $\oplus$ sembolü ile gösterilir.

Doğruluk Tablosu (Truth Table)

XOR'un temel kuralı şudur: **Bitler birbirinden farklıysa sonuç 1, aynıysa 0'dır.**

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Örnek Uygulama:

İki binary sayıyı (1010 ve 1100) XOR'layalım:

Plaintext

```
  1010
⊕ 1100
-----
  0110
```

(Sırasıyla: $1 \oplus 1 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $0 \oplus 0 = 0$)

Kriptografik Özellikleri ve Mantığı

XOR'u "mükemmel" yapan bazı matematiksel özellikler vardır:

- **Kendisiyle XOR:** Herhangi bir değer kendisiyle XOR'lanırsa sonuç 0 olur ().
- **Sıfır ile XOR:** Bir değeri 0 ile XOR'lamak o değeri değiştirmez ().
- **Değişme ve Birleşme Özelliği:** İşlem sırası sonucu değiştirmez ().

XOR ile Simetrik Şifreleme Mantığı

XOR, en basit simetrik şifreleme algoritması olarak kullanılabilir. Diyelim ki elimizde bir mesaj (**P - Plaintext**) ve gizli bir anahtar (**K - Key**) var:

1. **Şifreleme:** (Mesajı anahtar ile XOR'la ve şifreli metni elde et).
2. **Şifre Çözme:** (Şifreli metni aynı anahtar ile tekrar XOR'la).

Neden çalışır?

Matematiksel ispatı şöyledir:

Birleşme özelliğinden dolayı:

olduğu için:

Sonuç: Orijinal mesaj geri geldi!

2. Modulo İşlemi (Kalan Bulma)

Kriptografide sıkça karşımıza çıkan $\%$ veya `mod` operatörü, bir sayının diğerine bölünmesinden kalan sonucu verir. Günlük hayatta saatin 13:00'ten 2 saat sonra 15:00 değil de (12 saatlik formatta) 03:00 olması bir modulo işlemidir.

Örnekler

- $25 \% 5 = 0$ (Çünkü 25, 5'e tam bölünür, kalan 0'dır).
- $23 \% 6 = 5$ (Çünkü $23 = 3 \cdot 6 + 5$).
- $23 \% 7 = 2$ (Çünkü $23 = 3 \cdot 7 + 2$).

Kriptografi İçin Neden Önemli?

1. **Geri Döndürülemezlik (Irreversibility):** Eğer size dersem, 'in ne olduğunu kesin olarak bilemezsiniz. 9 olabilir, 14 olabilir, 19 olabilir... Bu özellik, tek yönlü fonksiyonlar oluşturmak için kritiktir.
2. **Sınırlandırma:** işleminin sonucu her zaman ile -1 arasındadır. Bu, verinin belirli bir aralıkta (örneğin alfabedeki harf sayısı olan 26 içinde) kalmasını sağlar.

Teknik İpuçları ve Araçlar

Büyük sayılarla (örneğin RSA'daki gibi yüzlerce haneli sayılarla) işlem yaparken standart hesap makineleri hata verebilir veya hassasiyeti kaybedebilir. Bu durumda şunlar kullanılır:

- **Python:** Hiçbir ek kütüphaneye gerek duymadan devasa tam sayıları (`arbitrary-precision integers`) işleyebilir.

```
| print(2**1024 % 13) komutu saniyeler içinde doğru sonucu verir.
```

- **WolframAlpha:** Karmaşık modüler aritmetik denklemleri çözmek için en iyi online araçtır.
- **BigInteger Kütüphaneleri:** C++ veya Java gibi dillerde bu tür işlemler için özel kütüphaneler (örneğin Java'da `BigInteger` sınıfı) kullanılmalıdır.